

# PyGPT

A Python code-completion assistant, trained from scratch

Every stage built and measured by hand: data → tokenizer → model → training → evaluation → deployment

Rehan · Machine Learning · Bennett University · October 2026

# The problem and the end goal

- Task: given the start of a Python function, predict what comes next — next-token prediction is code completion
- Goal: a working assistant powered by a model we trained ourselves — not an API call, not a fine-tune
- “Working” means three things:
  - a live demo that completes code in seconds
  - measured results against baselines (perplexity, syntactic validity, functional tests)
  - a public repo whose structure matches the design in this deck
- Constraints: 3 days, free compute only (Kaggle 2× T4, 30 GPU-h/week), solo

# Existing systems — and the gap

## Criterion 1

| System                           | Year    | Params       | Open? | Relevance                                                  |
|----------------------------------|---------|--------------|-------|------------------------------------------------------------|
| GPT-2 (Radford et al.)           | 2019    | 124M–1.5B    | Yes   | Architecture we reuse; English-only → our baseline         |
| Codex / Copilot (Chen et al.)    | 2021    | 12B          | No    | Defined the task and pass@k evaluation                     |
| CodeParrot (HF)                  | 2021    | 110M / 1.5B  | Yes   | Closest prior work: same data family, 50–100× our compute  |
| SantaCoder / StarCoder (BigCode) | 2022–23 | 1.1B / 15.5B | Yes   | License-filtered data pipeline we follow in spirit         |
| Code Llama (Meta)                | 2023    | 7B–70B       | Yes   | Open state of the art                                      |
| PyGPT (ours)                     | 2026    | 110.5M       | Yes   | Complete, reproducible, measured pipeline at student scale |

*Gap: everything above is closed-source or needs 100–10,000× a student's compute. None of it teaches how a code LM is built. PyGPT's contribution is honest numbers on what ~7 GPU-hours buys — with every stage visible.*

# Feasibility: do the math before burning GPU hours

## Criterion 1

- Scaling-law estimate:  $\text{FLOPs} \approx 6 \cdot N \cdot D = 6 \times 1.1\text{e}8 \times 8\text{e}8 \approx 5.3\text{e}17 \rightarrow \sim 6 \text{ h}$  on 2× T4 at 20–30% utilization
- Measured on the day: 34,000 tok/s (fp16, DDP)  $\rightarrow$  799.5M tokens in 6 h 45 min — one full epoch
- Chinchilla-optimal for 110M params  $\approx$  2.2B tokens  $\rightarrow$  we are  $\sim 3\times$  under-trained by design (limitations slide)
- The free-tier landscape is a moving target — verify, don't assume:
  - AWS: GPU quota denied twice (new account)  $\rightarrow$  dead end
  - Hugging Face: free Gradio Spaces discontinued mid-project (402)  $\rightarrow$  laptop demo instead
  - Kaggle: 30 GPU-h/week free — final 2.35B-token run took 17.5 h across two resumed sessions, inside one weekly quota
- Session-death insurance: checkpoint every 250 steps + tested kill-and-resume before the long run

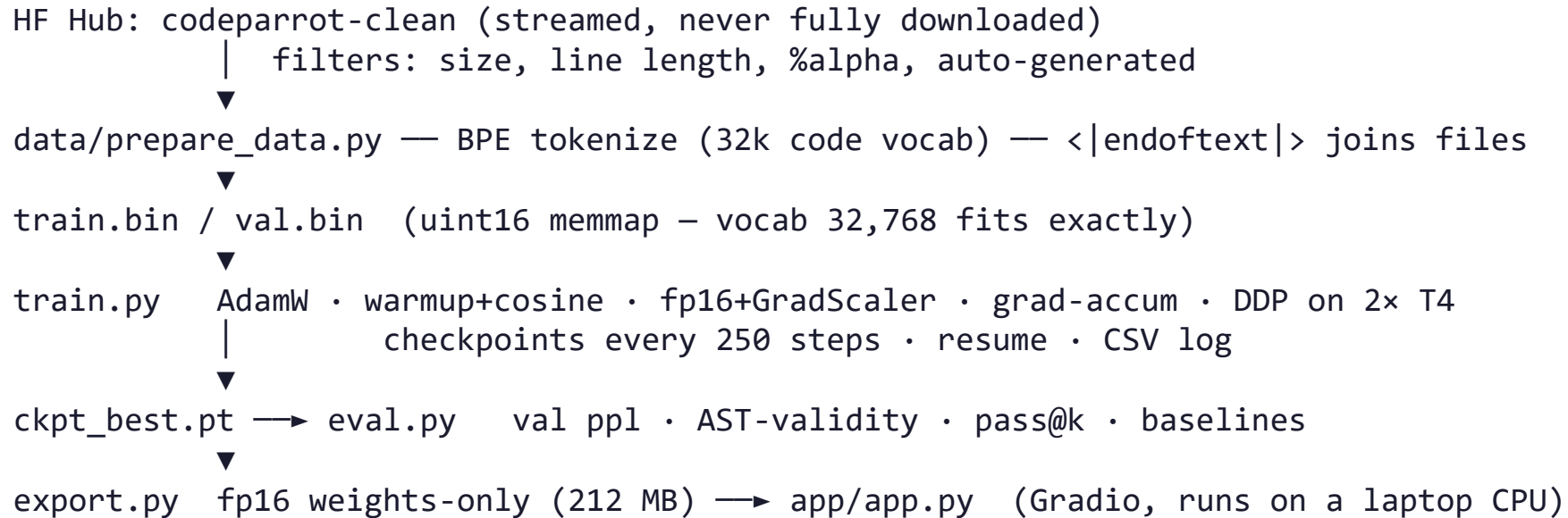
# Objectives — all measurable, all measured

Criterion 2

| #  | Objective                                    | Target                                                 | Achieved                                                                                               |
|----|----------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| O1 | GPT-2-class model from scratch in PyTorch    | init loss $\approx \ln(32768)=10.4$ ; causal-mask test | 10.42 ✓ · mask test ✓ · 110.5M params ✓                                                                |
| O2 | Clean Python pretraining corpus              | $\geq 800\text{M}$ train tokens + held-out val         | 800M + 5M (codeparrot-clean) ✓                                                                         |
| O3 | Useful loss on free GPUs                     | val loss $< 1.8$ in $\leq 10$ GPU-h                    | 1.509 — 2.35B fresh tokens, $\approx$ Chinchilla-optimal ✓ (ppl 4.52)                                  |
| O4 | Quantitative eval vs baselines               | ppl, AST-validity, pass@k                              | AST-valid 60% vs GPT-2 7% ✓ · DSA pass@10 0.80 vs GPT-2 0.00 ✓                                         |
| O5 | Deployed completion app                      | 30-token prompt answered $< 15$ s                      | 2.4 s for 96 tokens on laptop CPU ✓                                                                    |
| O6 | Design $\leftrightarrow$ implementation sync | map table + public repo                                | slide 11 + GitHub + <a href="https://huggingface.co/rehannn11223/pygpt">hf.co/rehannn11223/pygpt</a> ✓ |
| O7 | (stretch) VS Code extension                  | 60-s video                                             | future work                                                                                            |

# Methodology: the pipeline

Criterion 2 — every box is a file in the repo



# Architecture: GPT-2 Small, written from scratch

Criteria 3+4 — file and class names on every box

```
tokens → wte (32768×768) + wpe (512×768)      model.py: GPT
      ▼
12 × Block:  x = x + CausalSelfAttention(LN(x))  model.py: Block, CausalSelfAttention
              x = x + MLP(LN(x))                model.py: MLP    (768 → 3072 → 768, GELU)
      ▼
LayerNorm → lm_head (tied to wte)              weight tying: -25M params
```

- Parameters: embeddings 25.6M + 12 blocks × 7.1M ≈ 110.5M (“124M-class”: GPT-2's 50k English vocab vs our 32k code vocab)
- Why each choice: pre-LN → stable at 12 layers · SDPA kernel → memory-efficient attention on T4 · ctx 512 → 1.4× faster, functions fit
- Generation: temperature / top-k / nucleus sampling + per-layer KV cache (each step feeds 1 token, not the whole sequence)

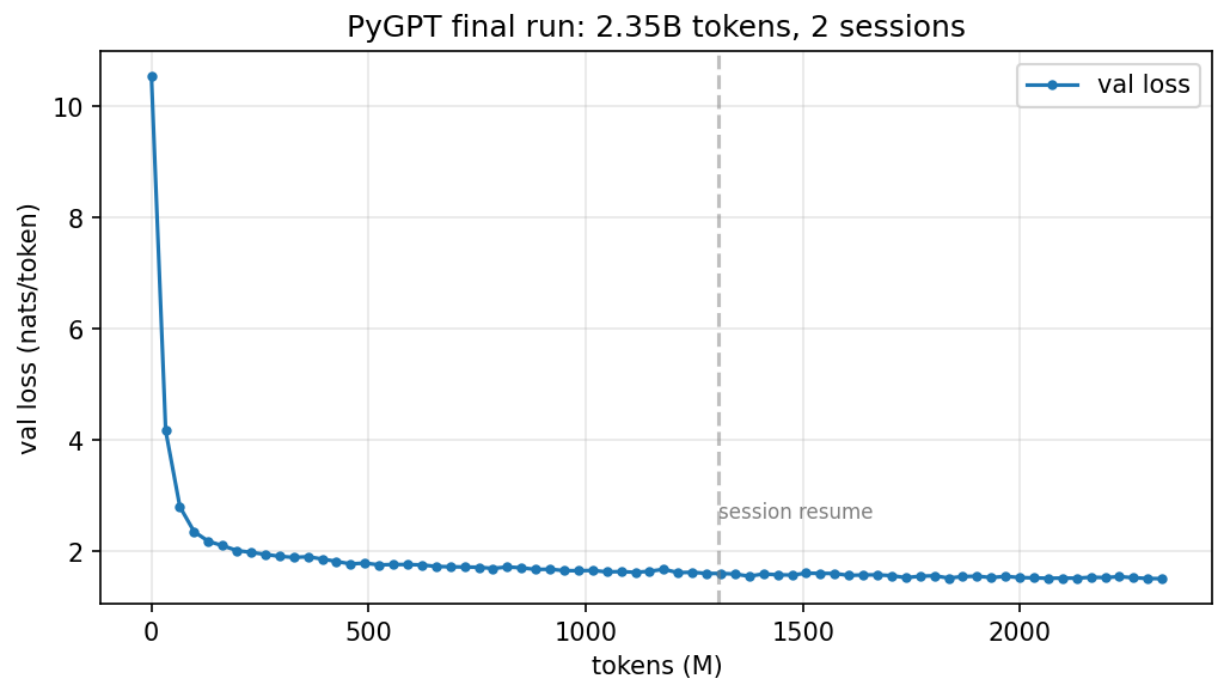
# Data, tokenizer, training recipe — each row has a why

Criterion 3

| Choice        | Value                                                                 | Why                                                                                                                                                                    |
|---------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data          | codeparrot-clean, 800M tokens streamed                                | pre-deduplicated Python; streaming avoids a 50 GB download                                                                                                             |
| Tokenizer     | codeparrot BPE, vocab 32,768                                          | code-trained: indentation + identifiers are single tokens (~35% fewer than GPT-2's)                                                                                    |
| Optimizer     | AdamW $\beta=(0.9, 0.95)$ , wd 0.1 on matrices only                   | decoupled decay regularizes without fighting Adam                                                                                                                      |
| LR schedule   | warmup 200 $\rightarrow$ 6e-4 $\rightarrow$ cosine $\rightarrow$ 6e-5 | warmup avoids early divergence; low final LR sharpens the minimum                                                                                                      |
| Precision     | fp16 autocast + GradScaler                                            | 2–3 $\times$ throughput on T4 tensor cores; scaler prevents underflow                                                                                                  |
| Batching      | 16 $\times$ 512 $\times$ 2 GPUs $\times$ grad-accum 8 = 131k tok/step | large effective batch within 16 GB                                                                                                                                     |
| Multi-GPU     | DistributedDataParallel (torchrun)                                    | near-linear scaling on the 2 free T4s                                                                                                                                  |
| Safety        | grad clip 1.0 $\cdot$ ckpt/250 steps $\cdot$ tested resume            | fp16 spikes; Kaggle 12 h session cap                                                                                                                                   |
| Prompt format | prompts end at the closing "" — no trailing newline                   | newline+indent is ONE BPE token; a bare <code>\n</code> says “dedent next” and the model closes the function. Removing one character: pass@10 went 0 $\rightarrow$ 0.4 |

# Results — against real baselines

Criteria 2+3 · all numbers in results/metrics.json



| Metric                  | Random init | GPT-2 124M        | PyGPT (ours) |
|-------------------------|-------------|-------------------|--------------|
| Val perplexity (Python) | 38,487      | n/a (other vocab) | 4.52         |
| AST-valid samples       | 0%          | 7%                | 60%          |
| pass@1 (5 DSA tasks)    | —           | 0.00              | 0.60         |
| pass@10 (5 DSA tasks)   | —           | 0.00              | 0.80         |

Final run: 10.4 → 1.509 over 2.35B fresh tokens (≈Chinchilla-optimal) with the DSA curriculum blended at 6.6% — no specialization tax this time; one LR schedule across two sessions (dashed line = tested resume).

pass@10 journey: 0.00 → 0.39 (removing ONE prompt character — the newline/BPE finding, slide 8) → 0.58 (bolt-on curriculum) → 0.80 (fresh data + blend). GPT-2: 0.00 throughout. add: 30/30, divide: 29/30 at n=30.

is\_prime: 0 passes for every model — the honest boundary of 110M params at this budget.

# Live demo

Criterion 4

- `python app/app.py` — runs offline on this laptop, ~2.4 s per completion (KV cache)
- Prompt A: `def fibonacci(n) + docstring` — read what it does, not what we hope it does
- Prompt B: `NumPy normalize()` — has it learned library idioms?
- Prompt C: `class Stack with __init__` — multi-method structure
- Same prompt at temperature 0.2 vs 0.9 — sampling in one sentence
- Prompt F: `solve_sudoku()` — the honest failure case
  - backups, in order: Kaggle public share link · recorded video · screenshots

# Design ↔ implementation: nothing on these slides is vaporware

Criterion 4

| Design component              | File · symbol                                     | Verified by                                                                      |
|-------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------|
| Data filtering + tokenization | data/prepare_data.py · keep_file, tokenize_stream | token counts re-counted from disk; decoded sample inspected                      |
| Causal self-attention         | model.py · CausalSelfAttention                    | test: changing token $t+1$ never changes logits $\leq t$                         |
| Full model + init             | model.py · GPT                                    | init loss $10.42 \approx \ln(V)$ ; overfits one batch to $< 0.1$ ; 110.5M params |
| Training loop                 | train.py · get_lr, estimate_loss, save_ckpt       | smoke run · kill + --resume continues · DDP on 2 GPUs · time-limit exit          |
| Sampling + KV cache           | model.py · generate                               | cached output bit-identical to uncached (sanity check 5)                         |
| Evaluation                    | eval.py · val_loss, ast_validity, functional_test | results/metrics.json — the numbers on slide 9                                    |
| Export + demo                 | export.py, app/app.py                             | fp16 round-trip greedy-identical; this demo                                      |

Code: [github.com/Rehan123-bash/pygpt](https://github.com/Rehan123-bash/pygpt) · Weights + runnable demo: [huggingface.co/rehannn11223/pygpt](https://huggingface.co/rehannn11223/pygpt)

# Limitations and future work

Criterion 1 — knowing the boundary is part of the result

- Under-trained  $\sim 3\times$  vs Chinchilla-optimal (0.8B of  $\sim 2.2$ B tokens) — loss was still falling when we stopped
- Completion  $\neq$  correctness:  $\text{pass}@10 = 0$  on toy tasks; it autocompletes plausibly, it does not reason
- 512-token context — function-level, not file-level, completion
- No instruction tuning: “write a function that...” prompts will not work
- Specialization tax: the DSA curriculum cost +0.18 nats of general-Python loss — narrow gains aren't free
- Future work:
  - longer training — 22 unused free GPU-hours per week were left on the table
  - train our own tokenizer · fill-in-the-middle objective · VS Code inline-completion extension (backend already exists)

# References

- Vaswani et al. 2017 — Attention Is All You Need
- Radford et al. 2019 — Language Models are Unsupervised Multitask Learners (GPT-2)
- Chen et al. 2021 — Evaluating LLMs Trained on Code (Codex, HumanEval, pass@k)
- Tunstall, von Werra, Wolf 2022 — NLP with Transformers, ch. 10 (CodeParrot)
- Kocetkov et al. 2022 — The Stack · Li et al. 2023 — StarCoder
- Rozière et al. 2023 — Code Llama
- Kaplan et al. 2020 — Scaling Laws · Hoffmann et al. 2022 — Chinchilla
- Sennrich et al. 2016 — BPE · Press & Wolf 2017 — Weight Tying
- Loshchilov & Hutter 2019 — AdamW · Micikevicius et al. 2018 — Mixed Precision
- Holtzman et al. 2020 — The Curious Case of Neural Text Degeneration (nucleus sampling)
- Karpathy — nanoGPT (reference implementation pattern)